ByteMart Digital

THE INTERVIEW PREP SERIES

Reactive Microservices Interview Q&A

Reactive foundations, RSocket, messaging, resilience, data, gateway & observability

116 interview questions with concise, accurate model answers
challenges with working solutions — organized by topic for focused preparation.

116	0	16	Model
Questions	Coding	Sections	Answers

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Reactive Microservices Interview Questions

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
This material is provided in good faith for educational and interview-preparation purposes. While every effort has been made to ensure technical accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from its use. All code examples are provided "as is".

Trademarks
Java is a registered trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. All other product names and brands are the property of their respective owners and are used for identification purposes only. This is an independent work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

Reactive Fundamentals 4

Reactive Communication 7

Reactive Messaging 10

Resilience 12

Data & Consistency 15

Gateway & Discovery 17

Observability 19

Deployment & Scaling 21

Patterns & Testing 23

RSocket Deep-Dive 25

Reactive Kafka & Messaging 27

Reactive Security 29

Reactive vs Alternatives 31

Config, Discovery & Performance 33

Caching, Events & Streaming 35

Migration & Scenarios 37

This bank contains 116 Reactive Microservices interview questions across 16 sections. Use the bookmarks panel to jump to any section or question.

SECTION 01

Reactive Fundamentals

Q1 What is a reactive microservice?

ANSWER

A reactive microservice is a microservice built on a non-blocking, asynchronous foundation — using reactive streams end to end (reactive web layer, reactive clients, reactive data access, and message-driven communication). It handles high concurrency with few threads, propagates backpressure, and is designed to stay responsive under load and partial failure.

> **KEY IDEA**
> Reactive microservices apply reactive programming across the whole service so it scales on I/O concurrency and remains responsive and resilient under load.

Q2 What are the four traits of the Reactive Manifesto?

ANSWER

The Reactive Manifesto describes systems that are responsive, resilient, elastic, and message-driven. The first three are goals; the last is the means to achieve them.

Trait	Meaning
Responsive	reacts in a timely manner
Resilient	stays responsive under failure
Elastic	scales with varying load
Message-driven	async messages provide the foundation

Q3 Why do reactive microservices scale better under high concurrency?

ANSWER

Traditional services use a thread per request, so concurrency is capped by the thread pool and slow I/O ties up threads. Reactive services never block on I/O, so a small event-loop pool serves thousands of concurrent connections. When a service fans out to many slow dependencies, the reactive model keeps threads free instead of parking them.

© 2026 ByteMart Digital

Q4 What is the C10k / thread-economy problem?

ANSWER

The C10k problem is handling ten thousand (or far more) concurrent connections. Thread-per-request models fail here because threads are expensive (memory, context switching) and mostly sit idle waiting on I/O. Reactive, event-loop-based services decouple connections from threads, so concurrency is bounded by memory and event-loop capacity, not thread count.

Q5 How do reactive microservices differ from traditional ones?

ANSWER

Architecturally they follow the same principles — bounded contexts, database per service, independent deployment. The difference is the execution model: reactive services are non-blocking throughout, favour asynchronous message-driven communication, and propagate backpressure across boundaries. Traditional services are typically blocking and synchronous-first.

	Traditional	Reactive
Threading	thread per request	event loop
Communication	sync-first (REST)	message-driven, streaming
Backpressure	none	end to end

Q6 What does it mean to be message-driven, and why does it matter?

ANSWER

Message-driven means services communicate primarily by passing asynchronous messages rather than blocking request/response calls. This decouples services in time and location, provides natural load-levelling (the broker buffers bursts), enables backpressure, and localises failures — a down consumer simply catches up later rather than failing the caller.

Q7 What is backpressure in a distributed, cross-service context?

ANSWER

Backpressure across services means a downstream service (or consumer) can signal that it is overwhelmed, so upstream producers slow down instead of flooding it. It is realised through demand-based protocols (RSocket, Reactive Streams over the wire) and, with brokers, through consumer-paced consumption — preventing cascading overload.

© 2026 ByteMart Digital

Q8 When should you NOT build reactive microservices?

ANSWER

Reactive adds real complexity — a harder programming model, tricky debugging, and the requirement that every I/O layer be non-blocking. Avoid it for CPU-bound services, systems tied to blocking drivers (JDBC/JPA with no reactive path), small low-traffic services, or teams new to reactive, where the added complexity outweighs the concurrency benefit.

> **WATCH OUT**
> Half-reactive systems are a trap — one blocking layer negates the benefit while keeping the complexity. Go reactive end to end, or stay imperative.

SECTION 02

Reactive Communication

Q9 What are the communication options between reactive microservices?

ANSWER

Reactive services communicate via non-blocking HTTP (`WebClient` returning `Mono/Flux`), asynchronous messaging (reactive Kafka or RabbitMQ), and **RSocket**, a binary protocol with native backpressure and streaming. The choice depends on whether you need request/response, streaming, or fully decoupled events.

Q10 How does non-blocking HTTP communication work between services?

ANSWER

Services call each other with `WebClient`, which returns reactive types and never blocks the calling thread. Because calls are non-blocking, a service can fan out to many dependencies concurrently and combine results with `zip`, keeping latency close to the slowest call rather than the sum of all calls.

```java
Mono.zip(inventoryClient.check(id), pricingClient.quote(id))
    .map(t -> new Offer(t.getT1(), t.getT2()));   // concurrent, non-blocking
```

Q11 Why is asynchronous messaging preferred for reactive systems?

ANSWER

Synchronous call chains couple service availability and cascade failures. Asynchronous messaging decouples services in time: the broker buffers load, consumers process at their own pace with backpressure, and a temporarily-down service catches up later. This aligns with the message-driven principle and maximises resilience and elasticity.

Q12 What is RSocket?

ANSWER

RSocket is a binary application protocol for reactive communication with **backpressure built into the wire**. Unlike REST, it supports four interaction models, multiplexes many streams over one connection, works over TCP or WebSocket, and can resume broken connections — designed specifically for reactive, streaming service-to-service communication.

© 2026 ByteMart Digital

Q13 What are RSocket's four interaction models?

ANSWER

Model	Semantics
Request-Response	one request, one response
Request-Stream	one request, many responses
Fire-and-Forget	one request, no response
Channel	bidirectional streams

This richer set (beyond REST's request/response) lets services model streaming and bidirectional flows natively, each with backpressure.

Q14 How does RSocket compare to REST and gRPC?

ANSWER

REST is text-based request/response over HTTP with no backpressure. gRPC adds streaming and binary Protobuf over HTTP/2 but treats backpressure as a lower-level concern. RSocket makes reactive semantics and application-level backpressure first-class across all four interaction models, making it the most natural fit for fully reactive systems — at the cost of less ubiquity than REST.

Q15 How is backpressure propagated across the network?

ANSWER

With RSocket, the consumer's demand (`request(n)`) is carried over the wire, so a producer on another service only sends what the consumer requested. With brokers like Kafka, backpressure is achieved by consumers pulling at their own pace. Plain REST cannot propagate backpressure, which is why reactive systems favour RSocket or messaging for streaming.

Q16 What is event-driven communication, and how does it fit reactive systems?

ANSWER

In event-driven communication, services publish immutable events about what happened and others react, with no direct coupling. It fits reactive systems perfectly: events flow as asynchronous message streams with backpressure, services stay decoupled and independently scalable, and the system tolerates partial failure gracefully.

© 2026 ByteMart Digital

Q17 How do you stream data continuously between two services?

ANSWER

Use a streaming-capable channel — an RSocket request-stream, a `Flux` over server-sent events, or a message topic consumed reactively. The producer emits a `Flux` and the consumer processes elements as they arrive with backpressure, so neither side buffers the whole dataset.

SECTION 03

Reactive Messaging

Q18 What is Reactor Kafka?

ANSWER

Reactor Kafka is a reactive API over the Kafka client that exposes producing and consuming as `Flux`-based streams with backpressure. Consumers pull records at their own pace and process them non-blockingly, so a slow consumer naturally throttles consumption instead of falling behind uncontrollably.

```java
KafkaReceiver.create(receiverOptions)
    .receive()
    .flatMap(record -> handle(record.value())
        .doOnSuccess(v -> record.receiverOffset().acknowledge()));
```

Q19 How does backpressure work with a message broker?

ANSWER

A broker decouples producer and consumer speeds by persisting messages. The consumer requests and processes messages at its own rate, and reactive consumers extend this by only pulling what the downstream pipeline demands. The broker itself acts as a buffer, absorbing bursts — a form of load-levelling backpressure.

Q20 What is Spring Cloud Stream, and does it support reactive?

ANSWER

Spring Cloud Stream is a framework for building message-driven microservices with a binder abstraction over Kafka, RabbitMQ, and others. It supports a reactive functional model where handlers are `Function<Flux, Flux>`, letting you process message streams reactively with backpressure while staying broker-agnostic.

```java
@Bean
public Function<Flux<Order>, Flux<Invoice>> process() {
    return orders -> orders.flatMap(this::toInvoice);
}
```

Q21 How do you guarantee at-least-once processing reactively?

ANSWER

Acknowledge (commit) the message offset only **after** the reactive processing pipeline for that message completes successfully. Because acknowledgement is tied to the completion signal, a failure before completion leaves the message unacknowledged and it is redelivered — giving at-least-once semantics.

Q22 Why must reactive consumers be idempotent?

ANSWER

At-least-once delivery means a message may be processed more than once (on retry or redelivery). Idempotent consumers — keyed on a message id or a natural unique constraint — ensure reprocessing a duplicate has no additional effect, which is what makes at-least-once safe in practice.

Q23 What is a dead letter queue in a reactive pipeline?

ANSWER

A dead letter queue captures messages that repeatedly fail processing after the retry limit, so one poison message does not stall the consumer. In a reactive flow, after exhausting `retryWhen`, you route the failed message to the DLQ within `onErrorResume` and continue consuming.

Q24 How does event sourcing work in a reactive system?

ANSWER

Event sourcing stores state changes as an append-only stream of events, which is naturally reactive: the event log is a `Flux` that projections consume to build read models. Reactive stream processing rebuilds and updates projections non-blockingly, and the event stream can be replayed to reconstruct state.

Q25 What is the transactional outbox pattern, and how does it apply reactively?

ANSWER

To publish an event reliably alongside a database change (avoiding the dual-write problem), write the event to an outbox table in the same local transaction, then a separate reactive process streams the outbox to the broker. This guarantees the event is published if and only if the data was committed — the reactive relay reads the outbox as a `Flux`.

© 2026 ByteMart Digital

SECTION 04

Resilience

Q26 How do you apply a circuit breaker in a reactive service?

ANSWER

Use Resilience4j's reactive operators, which wrap a `Mono`/`Flux` with circuit-breaking behaviour. When failures cross a threshold the circuit opens and calls fail fast (or fall back) without blocking, giving the dependency time to recover — all composed into the reactive chain.

```java
return webClient.get().uri(uri).retrieve().bodyToMono(Data.class)
    .transformDeferred(CircuitBreakerOperator.of(circuitBreaker))
    .onErrorResume(e -> Mono.just(Data.fallback()));
```

Q27 How do timeouts work reactively, and why are they essential?

ANSWER

The `timeout(Duration)` operator errors (or switches to a fallback) if no response arrives in time, without blocking a thread. Timeouts are essential because a slow dependency must not hold resources indefinitely; in reactive systems the timeout frees the pipeline to fall back or fail fast.

Q28 How do you implement retries with backoff reactively?

ANSWER

Use `retryWhen(Retry.backoff(maxAttempts, initialDelay))`, which resubscribes with exponential backoff and jitter, optionally filtered to transient errors. Because retries are non-blocking and only re-run idempotent operations, they add resilience without tying up threads.

```java
call().retryWhen(Retry.backoff(3, Duration.ofMillis(200))
    .filter(ex -> ex instanceof TransientException)
    .jitter(0.5));
```

© 2026 ByteMart Digital

Q29 What is the bulkhead pattern in a reactive context?

ANSWER

A bulkhead isolates resources per dependency so one slow or failing dependency cannot exhaust everything. Reactively, Resilience4j's bulkhead limits concurrent calls to a dependency (and `flatMap`'s concurrency parameter caps in-flight work), so a struggling dependency's problems stay contained.

Q30 How does backpressure itself act as a resilience mechanism?

ANSWER

Backpressure prevents overload at the source: because a consumer only accepts what it can handle, a fast producer or a burst cannot flood and crash it. This built-in flow control means reactive systems degrade gracefully under load rather than collapsing — resilience by design rather than by add-on.

Q31 How do you provide a graceful fallback reactively?

ANSWER

Use `onErrorResume` to switch to an alternative publisher — cached data, a default, or a degraded response — when the primary call fails or the circuit is open. The service keeps serving a useful (if reduced) result instead of surfacing an error, all without blocking.

Q32 How do reactive systems prevent cascading failures?

ANSWER

They combine timeouts (never wait forever), circuit breakers (fail fast on unhealthy dependencies), bulkheads (isolate resources), and backpressure (reject or slow excess work). Crucially, because threads are never blocked on I/O, a slow dependency does not exhaust a thread pool and stall the whole service — removing the main vector of cascade.

> **KEY IDEA**
> Non-blocking I/O plus timeouts, circuit breakers, bulkheads, and backpressure together stop one failing dependency from taking down the system.

Q33 What is a rate limiter, and how is it applied reactively?

ANSWER

A rate limiter caps how many calls are allowed per time window, protecting a dependency or the service itself from overload. Resilience4j provides a reactive rate-limiter operator that delays or rejects calls exceeding the limit, composed into the pipeline without blocking.

© 2026 ByteMart Digital

SECTION 05

Data & Consistency

Q34 How do reactive microservices access their databases?

ANSWER

Each service owns its data and accesses it with a **non-blocking driver** — R2DBC for relational stores, reactive MongoDB/Cassandra/Redis for others. Repositories return `Mono/Flux`, so data access composes into the reactive pipeline without blocking event-loop threads.

> **WATCH OUT**
> A blocking JDBC/JPA layer breaks a reactive service — database-per-service still applies, but each service needs a reactive driver to stay non-blocking end to end.

Q35 How do you handle distributed transactions reactively?

ANSWER

As with any microservices system, you avoid distributed ACID transactions and use the **Saga** pattern — a sequence of local transactions with compensating actions. Reactively, each step is a `Mono` chained with `flatMap`, and a failure triggers compensations via `onErrorResume`, all non-blocking.

Q36 How is a Saga implemented reactively?

ANSWER

An orchestrated Saga chains the steps as reactive operations; if a later step fails, the chain runs compensating operations for the completed steps. Choreographed Sagas instead react to events on a stream. Either way, the reactive model expresses the sequencing and compensation as composed publishers.

```java
return orderService.create(req)
    .flatMap(order -> paymentService.charge(order)
        .flatMap(p -> inventoryService.reserve(order)
            .onErrorResume(e -> paymentService.refund(order)
                .then(Mono.error(e))))
        .onErrorResume(e -> orderService.cancel(order)
            .then(Mono.error(e))));
```

Q37 What is eventual consistency, and why is it inherent here?

ANSWER

Because each service owns its data and they synchronise through asynchronous events, the copies of related data across services are consistent only eventually — there is a brief window of divergence. Reactive, message-driven systems embrace this trade-off for availability and decoupling, and design UX and logic to tolerate short staleness.

Q38 How does CQRS fit a reactive architecture?

ANSWER

CQRS separates the write model from read models. Reactively, commands update the write side and emit events; a stream processor consumes those events (a `Flux`) to maintain denormalised read models. Reads then serve pre-computed views quickly, and the event-driven projection keeps them eventually consistent — a natural reactive pipeline.

Q39 How do you keep a local read replica updated across services?

ANSWER

A service that needs another's data subscribes to that service's event stream and maintains a local, read-only copy updated reactively as events arrive (event-carried state transfer). This removes a synchronous runtime dependency, improving availability at the cost of eventual consistency of the replica.

Q40 How do you coordinate reactive writes within one service transactionally?

ANSWER

Chain the writes with `flatMap/then` and wrap the method in a reactive transaction (`@Transactional` with a `ReactiveTransactionManager`, or `TransactionalOperator`). The transaction context flows through the Reactor `Context`, so the whole chain commits or rolls back atomically despite thread hops.

© 2026 ByteMart Digital

SECTION 06

Gateway & Discovery

Q41 Why is Spring Cloud Gateway a good fit for reactive systems?

ANSWER

Spring Cloud Gateway is built on WebFlux and Reactor Netty, so it is non-blocking by default. It routes and applies cross-cutting concerns (auth, rate limiting, circuit breaking) without a thread per request, handling high connection counts efficiently — exactly the edge behaviour a reactive system needs.

yaml

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: orders
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
          filters:
            - name: CircuitBreaker
              args: { name: ordersCb, fallbackUri: forward:/fallback }
```

Q42 How does the gateway apply rate limiting reactively?

ANSWER

The `RequestRateLimiter` filter (often backed by reactive Redis) enforces per-client limits at the edge without blocking, returning HTTP 429 when exceeded. Because it runs on the reactive stack, it can throttle many concurrent clients with minimal resources.

Q43 How is load balancing done in a reactive gateway?

ANSWER

Spring Cloud LoadBalancer provides client-side, reactive load balancing: the gateway resolves a service name (via discovery) to an instance and distributes requests non-blockingly. The `lb://` URI scheme in a route triggers this balanced resolution.

Q44 How does service discovery work with reactive services?

ANSWER

Reactive services register with a discovery mechanism (Eureka, Consul, or Kubernetes DNS) just like any microservice. The reactive load balancer and gateway resolve logical service names to instances at call time, so scaling and instance churn are handled transparently.

Q45 How do you apply circuit breaking at the gateway?

ANSWER

The gateway's `CircuitBreaker` filter wraps routes so that when a downstream service fails repeatedly, requests fail fast or route to a fallback URI — protecting the whole system at the edge. Running on the reactive stack, this adds no per-request thread cost.

Q46 What is a reactive Backend for Frontend (BFF)?

ANSWER

A reactive BFF is an edge service tailored to a client type that aggregates data from several downstream services non-blockingly — firing calls in parallel with `WebClient` and combining with `zip`. Its non-blocking nature makes fan-out aggregation efficient, returning a client-shaped response with low latency.

© 2026 ByteMart Digital

SECTION 07

Observability

Q47 Why is observability harder in reactive systems?

ANSWER

Reactive pipelines hop across threads and defer execution, so stack traces are unhelpful and `ThreadLocal`-based context (trace ids, MDC) does not follow the flow. Observability requires context propagation through the Reactor `Context` and reactive-aware instrumentation rather than the usual thread-bound tooling.

Q48 How is distributed tracing propagated across reactive services?

ANSWER

Micrometer Tracing, integrated with the context-propagation library, carries the trace and span ids through the Reactor `Context` and injects them into outbound `WebClient`/RSocket calls and messages. This reconstructs an end-to-end trace across services despite the thread hopping.

Q49 How do you keep trace ids in logs within a reactive flow?

ANSWER

Because MDC is `ThreadLocal`-based, bridge it into the reactive context with the Micrometer context-propagation hooks (`contextCapture`), so log lines emitted during a reactive operation carry the correct correlation id. Without this bridge, logs lose their trace context across thread switches.

Q50 How do you collect metrics from reactive microservices?

ANSWER

Micrometer instruments reactive streams (the `metrics()`/`name()` operators and reactive HTTP/RSocket instrumentation), and Spring Boot Actuator exposes them to Prometheus. Track the RED signals — rate, errors, duration — per endpoint and stream, plus event-loop and connection-pool metrics.

Q51 How do health checks work for reactive services?

ANSWER

Actuator provides reactive health indicators that check dependencies non-blockingly and aggregate into liveness and readiness endpoints. Because the checks are reactive, they compose the health of reactive data sources and downstream calls without blocking the event loop.

© 2026 ByteMart Digital

Q52 How do you debug a problem that spans reactive services?

ANSWER

Combine distributed traces (to see the cross-service timeline), correlation ids in centralised logs (to gather one request's logs), and per-service reactive debugging (`checkpoint`, the ReactorDebugAgent). Tracing locates the slow or failing service; the debug agent then makes that service's reactive stack trace readable.

SECTION 08

Deployment & Scaling

Q53 Why are reactive microservices more resource-efficient to run?

ANSWER

Because they serve many concurrent requests with few threads, reactive services need less memory (fewer thread stacks) and handle more load per instance. This can mean fewer instances for the same throughput — lower infrastructure cost — especially for I/O-bound, high-concurrency workloads.

Q54 How does the reactive model support elasticity?

ANSWER

Elasticity is scaling with load. Reactive services are typically stateless and efficient per instance, so they scale horizontally cleanly, and backpressure lets them shed or slow excess load gracefully while new instances spin up. This matches the Reactive Manifesto's elastic trait.

Q55 How do reactive services fit into Kubernetes?

ANSWER

They run as stateless containers that Kubernetes scales via the Horizontal Pod Autoscaler on CPU or custom (e.g. request-rate) metrics. Reactive readiness probes ensure traffic only reaches ready pods, and graceful shutdown drains in-flight reactive requests before termination.

Q56 What is the benefit of GraalVM native images for reactive services?

ANSWER

Compiling a reactive service to a GraalVM native image (via Spring's AOT support) gives near-instant startup and much lower memory use than the JVM. Combined with reactive efficiency, this suits serverless and rapidly-scaling deployments where fast, lightweight instances matter.

Q57 How does autoscaling interact with backpressure?

ANSWER

Backpressure lets each instance protect itself under load while the platform scales out. As load rises, backpressure slows intake and keeps instances stable; meanwhile autoscaling adds instances to raise total capacity. Together they provide smooth elasticity without individual instances collapsing.

© 2026 ByteMart Digital

Q58 What connection and thread settings matter most in production?

ANSWER

Key levers are the Netty event-loop thread count (default: CPU cores), the outbound connection pool size and pending-acquire limits for `WebClient`, response timeouts, and broker consumer concurrency. Right-sizing these prevents resource exhaustion and keeps the non-blocking advantage intact under load.

© 2026 ByteMart Digital

SECTION 09

Patterns & Testing

Q59 What is reactive API composition?

ANSWER

When a query needs data from several services, a composer calls them **in parallel** with non-blocking clients and combines the results. Reactively this is a `zip/flatMap` fan-out that keeps latency near the slowest call — the efficient way to aggregate across services without a shared database.

Q60 What is event-carried state transfer, and why does it suit reactive systems?

ANSWER

Events carry the data consumers need, so a consumer maintains its own local copy and avoids synchronous callbacks to the producer. This maximises decoupling and availability — and fits reactive systems because the events flow as backpressured streams that the consumer processes non-blockingly to update its state.

Q61 How do you test a reactive microservice?

ANSWER

Test in layers: `StepVerifier` for the reactive logic (asserting the signal sequence), `WebTestClient` for endpoints, Testcontainers for real reactive databases and brokers, and contract tests for the API. Virtual time tests delays and timeouts deterministically without waiting real seconds.

```java
StepVerifier.create(service.process(input))
    .expectNext(expected)
    .verifyComplete();
```

Q62 How do you test reactive messaging flows?

ANSWER

Use an embedded or Testcontainers broker so the reactive consumer runs against a real topic, publish test messages, and assert the resulting side effects (with `StepVerifier` or by polling the output). Verify idempotency by delivering duplicates and confirming a single effect.

© 2026 ByteMart Digital

Q63 What is contract testing, and why does it matter here?

ANSWER

Contract testing verifies that a service's API (REST, RSocket, or event schema) matches what consumers expect, catching breaking changes before deployment. In a reactive, event-driven system where services deploy independently and communicate via streams and events, contracts are what allow safe, independent evolution.

Q64 How do you apply chaos engineering to reactive services?

ANSWER

Inject faults — latency, dropped connections, killed instances, slow dependencies — and verify that timeouts, circuit breakers, backpressure, and fallbacks actually keep the system responsive. Chaos testing turns the assumed resilience of the reactive design into something continuously validated under real failure conditions.

Q65 What are the main anti-patterns in reactive microservices?

ANSWER

- **Hidden blocking** — a blocking driver or `block()` on an event-loop thread, silently killing scalability.
- **Half-reactive stacks** — reactive web layer over a blocking data layer, paying complexity for no gain.
- **Ignoring backpressure** — unbounded buffering that leads to out-of-memory failures.
- **Long synchronous chains** — recreating a distributed monolith instead of decoupling with events.
- **Losing context** — relying on `ThreadLocal` for trace/security data across thread hops.

> **WATCH OUT**
> The most damaging anti-pattern is hidden blocking — always verify the whole stack is non-blocking, ideally with BlockHound in tests.

SECTION 10

RSocket Deep-Dive

Q66 What transports does RSocket run over?

ANSWER

RSocket is transport-agnostic and runs over TCP, WebSocket, or Aeron. TCP suits low-latency service-to-service links; WebSocket works through browsers and firewalls that expect HTTP. The interaction models and backpressure behave identically regardless of transport.

Q67 How does routing work in RSocket?

ANSWER

RSocket carries **metadata** alongside each payload, and a routing metadata key names the destination handler (a route string, like a path). Spring maps these routes to `@MessageMapping` methods, so one connection can multiplex requests to many handlers.

```java
@MessageMapping("orders.create")
public Mono<Order> create(CreateOrder cmd) { ... }
```

Q68 What is connection resumption in RSocket?

ANSWER

RSocket can **resume** a session after a transient network drop, replaying missed frames instead of failing the in-flight streams. This makes long-lived streaming connections resilient to brief disconnects — valuable for mobile clients and unreliable networks.

Q69 When would you choose RSocket over messaging or REST?

ANSWER

Choose RSocket for low-latency, streaming, bidirectional service-to-service communication that needs wire-level backpressure — for example live data feeds or interactive streams. Use messaging (Kafka) for fully decoupled, durable event distribution, and REST for simple, ubiquitous request/response with broad tooling.

© 2026 ByteMart Digital

Q70 How is backpressure configured over RSocket?

ANSWER

Backpressure is intrinsic: the requester sends demand (leased or via `request(n)`) and the responder emits no more than requested. RSocket also supports **leasing**, where a responder advertises how many requests it can currently accept — letting an overloaded server throttle callers proactively.

Q71 How do you secure an RSocket connection?

ANSWER

Secure the transport with TLS, and authenticate at the RSocket layer by carrying credentials (a bearer token) in setup or per-request metadata, validated by reactive Spring Security. Authorization is enforced per route, just as with HTTP endpoints.

Q72 What is the fire-and-forget model good for?

ANSWER

Fire-and-forget sends a request with no expected response, ideal for high-volume, non-critical signals like metrics, telemetry, or notifications where you do not need confirmation. It is the most efficient interaction model because no response stream is established.

© 2026 ByteMart Digital

SECTION 11

Reactive Kafka & Messaging

Q73 How does offset management work in Reactor Kafka?

ANSWER

Each received record carries a `ReceiverOffset` that you acknowledge after successful processing. Acknowledgement is decoupled from consumption, so you commit only when the reactive pipeline for a record completes — the basis of reliable at-least-once processing.

Q74 How do consumer groups and partitions affect reactive consumers?

ANSWER

Kafka partitions a topic and assigns partitions across a consumer group for parallelism. A reactive consumer processes assigned partitions as streams; ordering is guaranteed only within a partition, so key related messages to the same partition when order matters, and scale throughput by adding partitions and consumers.

Q75 How do you achieve ordered processing with reactive Kafka?

ANSWER

Route related messages to the same partition (via a key) so they arrive in order, and process each partition's stream with an order-preserving operator such as `concatMap` rather than `flatMap`, which would interleave results.

Q76 How does producer-side backpressure work in reactive Kafka?

ANSWER

The reactive producer sends records as a `Flux` and receives send results as a stream; if the broker or buffer is saturated, demand slows the source. This prevents an unbounded backlog of in-flight sends from exhausting memory under a burst.

Q77 How do you handle poison messages reactively?

ANSWER

Bound retries with `retryWhen`, and on final failure route the message to a dead letter topic within `onErrorResume` before acknowledging it, so the consumer continues. This isolates the bad message without stalling the partition or losing it.

© 2026 ByteMart Digital

Q78 What is the difference between reactive Kafka and Spring Cloud Stream?

ANSWER

Reactor Kafka is a low-level reactive Kafka client giving fine control over records and offsets. Spring Cloud Stream is a higher-level, broker-agnostic framework where handlers are functions over message streams; it abstracts the broker and wiring, trading some control for simplicity and portability across Kafka, RabbitMQ, and others.

Q79 How do you scale a reactive message consumer?

ANSWER

Scale horizontally by adding consumer instances up to the partition count (each partition handled by one consumer in the group), and tune per-instance concurrency with a concurrency-capped `flatMap`. Backpressure ensures each instance only pulls what it can process.

SECTION 12

Reactive Security

Q80 How is authentication handled in reactive microservices?

ANSWER

Authentication is centralised at the edge (gateway), which validates a token and propagates it downstream. Each reactive service validates the token with reactive Spring Security and authorises from its claims. The security context flows through the Reactor `Context`, not a `ThreadLocal`.

Q81 How do you propagate a JWT across reactive service calls?

ANSWER

Forward the caller's bearer token on outbound `WebClient`/RSocket calls, typically via an `ExchangeFilterFunction` that reads the token from the reactive security context and adds the `Authorization` header. Each downstream service then validates it independently.

Q82 How does the reactive security context differ from the servlet one?

ANSWER

The servlet `SecurityContextHolder` is `ThreadLocal`-based and cannot survive reactive thread hops. Reactive Spring Security stores the context in the Reactor `Context` and exposes it via `ReactiveSecurityContextHolder`, so authorization works correctly across the asynchronous pipeline.

Q83 Why is mTLS important for reactive service-to-service calls?

ANSWER

Mutual TLS authenticates both ends of a connection, so only trusted services can communicate — essential in a zero-trust network. For reactive services (over HTTP or RSocket), mTLS is often provided transparently by a service mesh, securing all inter-service traffic without application code changes.

Q84 How do OAuth2 and OIDC apply to reactive services?

ANSWER

Reactive Spring Security supports OAuth2 resource-server and client configurations non-blockingly. A service validates access tokens (JWT) reactively and enforces scopes per endpoint, while a reactive OAuth2 client obtains and refreshes tokens for outbound calls — the same delegated-auth model, on the reactive stack.

© 2026 ByteMart Digital

© 2026 ByteMart Digital

SECTION 13

Reactive vs Alternatives

Q85 How does reactive compare to virtual threads (Project Loom)?

ANSWER

Virtual threads (Java 21+) make blocking code cheap by mapping many lightweight threads onto few OS threads, so you can write simple imperative, blocking-style code that still scales. Reactive achieves scalability through non-blocking streams with backpressure and rich composition. Loom removes much of reactive's concurrency motivation, but reactive still wins for streaming, backpressure, and declarative data-flow composition.

	Virtual threads	Reactive
Programming style	imperative, blocking	declarative streams
Backpressure	no	built in
Best for	simple high-concurrency	streaming + flow control

Q86 When might virtual threads be the better choice?

ANSWER

Prefer virtual threads when you want high concurrency with straightforward, readable imperative code and do not need streaming or backpressure — for example a service that makes several blocking calls per request. They give much of reactive's scalability with far less cognitive overhead, and work with existing blocking libraries.

Q87 How does the reactive model compare to the actor model?

ANSWER

The actor model (Akka) structures concurrency around isolated actors exchanging messages, each processing one message at a time. Reactive streams focus on data-flow pipelines with backpressure. They overlap in being message-driven and asynchronous; actors excel at stateful concurrency and supervision, reactive streams at composable, backpressured data processing.

Q88 Can reactive and blocking services coexist in one system?

ANSWER

Yes — microservices communicate over the network, so a reactive service and a blocking (MVC or virtual-thread) service interoperate via REST, messaging, or RSocket regardless of their internal models. This lets you adopt reactive selectively for the services that benefit, rather than rewriting everything.

© 2026 ByteMart Digital

Q89 Does reactive help CPU-bound workloads?

ANSWER

Not much — reactive's advantage is freeing threads during I/O waits, which CPU-bound work does not have. For CPU-heavy tasks, parallelism (a bounded pool, `ParallelFlux`, or virtual/platform threads) matters more than non-blocking I/O. Reactive can even add overhead without a concurrency benefit here.

SECTION 14

Config, Discovery & Performance

Q90 How is externalised configuration handled reactively?

ANSWER

Services load configuration from a central source (Spring Cloud Config, Consul, or Kubernetes ConfigMaps) at startup. The loading integrates with the reactive stack, and configuration refresh can be triggered so services pick up changes without a restart — keeping many instances consistent.

Q91 How do you refresh configuration without downtime?

ANSWER

Use a refresh mechanism (an actuator refresh endpoint or an event broadcast over the message bus) so instances re-read changed properties and rebind beans marked for refresh. This lets you roll out configuration changes across a fleet of reactive services with zero downtime.

Q92 What are the most important performance levers for a reactive service?

ANSWER

- **Event-loop threads** — default to CPU cores; rarely increase unless CPU-bound work is offloaded.
- **Connection pool size** — for outbound `WebClient` calls, sized to downstream capacity.
- **Timeouts** — on every outbound call to bound resource holding.
- **flatMap concurrency** — cap in-flight downstream calls.
- **Buffer / prefetch sizes** — balance throughput against memory.

Q93 How do you size the WebClient connection pool?

ANSWER

Match the pool to the downstream service's capacity and your concurrency needs — too small throttles throughput, too large can overwhelm the dependency and exhaust file descriptors. Set a pending-acquire timeout so requests fail fast rather than queueing indefinitely when the pool is saturated.

© 2026 ByteMart Digital

Q94 What is the danger of an unbounded buffer in a reactive service?

ANSWER

If a fast producer feeds a slow consumer through an unbounded buffer, the buffer grows until the service runs out of memory and crashes. Always bound buffers and choose an explicit overflow strategy (drop, latest, or error) so overload degrades gracefully instead of causing an out-of-memory failure.

> **WATCH OUT**
> Unbounded buffering is the classic way a reactive service silently accumulates memory until it dies — always bound and choose an overflow policy.

Q95 How do you implement load shedding reactively?

ANSWER

Reject or degrade excess work before it overwhelms the service — for instance a rate limiter that returns HTTP 429 past a threshold, or an `onBackpressureDrop` that discards surplus events. Shedding load keeps the service responsive for the requests it does accept, rather than slowing to a crawl for all.

© 2026 ByteMart Digital

SECTION 15

Caching, Events & Streaming

Q96 How do you cache in a reactive service?

ANSWER

Cache with a reactive store (reactive Redis) and compose it into the pipeline: try the cache, and on a miss fetch and populate it. The `cache()` operator can also memoise an expensive publisher in-process. Both avoid re-running costly work while staying non-blocking.

```java
return reactiveRedis.opsForValue().get(key)
    .switchIfEmpty(loadFromSource(key)
        .flatMap(v -> reactiveRedis.opsForValue().set(key, v).thenReturn(v)));
```

Q97 How do Sinks help publish inter-service events?

ANSWER

A multicast `Sinks.Many` lets application code push events into a shared `Flux` that can feed a message publisher or an SSE/RSocket stream. It bridges imperative event generation into a reactive stream that many subscribers or a broker adapter can consume with backpressure.

Q98 Can services communicate over WebSocket reactively?

ANSWER

Yes — a reactive `WebSocketHandler` supports full-duplex streaming between services or between a service and a browser client, exchanging `Fluxes` of messages. For pure service-to-service streaming, RSocket is usually preferred (native backpressure and interaction models), but WebSocket is useful when browser compatibility matters.

Q99 How do you fan out one event to many consumers?

ANSWER

Publish the event to a broker topic (Kafka) that all interested services subscribe to, or multicast in-process with a hot `Flux/Sinks`. The broker approach gives durability and independent consumer offsets; the in-process approach suits transient, same-service subscribers.

© 2026 ByteMart Digital

Q100 How do you process a stream in batches reactively?

ANSWER

Use `buffer(size)` or `bufferTimeout(size, duration)` to group elements into batches, then process each batch (e.g. a bulk database write). This amortises per-item overhead while keeping backpressure, and the timeout variant flushes partial batches so latency stays bounded.

```java
events.bufferTimeout(500, Duration.ofSeconds(1))
    .flatMap(batch -> repo.saveAll(batch).then());
```

Q101 How do you combine a request with a live stream of updates?

ANSWER

Emit the current snapshot first, then append the live update stream — using `concat` (snapshot Mono then updates Flux) or `startWith`. The client receives the initial state immediately and then continuous deltas over the same stream, a common pattern for dashboards.

SECTION 16

Migration & Scenarios

Q102 How do you migrate a blocking microservice to reactive?

ANSWER

Migrate incrementally with a strangler approach: introduce `WebClient` for outbound calls, convert endpoints to reactive types, and — crucially — replace blocking drivers with reactive ones (R2DBC, reactive Mongo). Until the data layer is reactive, isolate remaining blocking calls on `boundedElastic()`. Full benefit arrives only when the service is non-blocking end to end.

Q103 How do you verify no blocking calls remain after migration?

ANSWER

Run BlockHound in tests — it detects blocking calls on non-blocking threads and fails fast, surfacing the hidden JDBC call or `block()` that would otherwise silently stall the event loop. Combine with load tests that would expose thread starvation.

Q104 A service fans out to five downstreams and one is slow — how do you keep latency low?

ANSWER

Call all five in parallel with non-blocking clients and combine with `zip`, so total latency approaches the slowest call rather than the sum. Put a `timeout` and fallback on each, so a single slow or failing dependency degrades to a partial result instead of dragging the whole response.

```java
Mono.zip(a.call(), b.call(), c.call(), d.call(), e.call())
    .map(this::combine)
    .timeout(Duration.ofSeconds(2));
```

Q105 How would you design an order-processing flow across services reactively?

ANSWER

Model it as an event-driven Saga: the order service emits an `OrderCreated` event; payment, inventory, and shipping services react on their own streams and emit follow-up events; a failure emits a compensating event that unwinds prior steps. Each service processes its event stream reactively with idempotent, at-least-once handling.

Q106 How do you handle a downstream returning a huge result set?

ANSWER

Stream it rather than materialising it: consume the response as a `Flux` (streaming media type or RSocket request-stream), process elements as they arrive with bounded concurrency and `limitRate`, and avoid `collectList`. Memory stays bounded regardless of the total size.

Q107 How do you test resilience behaviour like timeouts and retries?

ANSWER

Use `StepVerifier.withVirtualTime` to advance the clock and assert timeout/retry behaviour deterministically, and Testcontainers with fault injection (toxiproxy) to simulate latency and failures against real dependencies. This validates that the circuit breaker, timeout, and fallback actually fire as designed.

Q108 What metrics tell you a reactive service is healthy under load?

ANSWER

- **Event-loop utilisation** — saturation signals blocking or CPU starvation.
- **Request rate, error rate, latency** (the RED signals) per endpoint.
- **Connection pool usage** and pending-acquire timeouts on outbound calls.
- **Backpressure indicators** — buffer sizes, dropped elements.
- **Circuit-breaker state** and retry counts per dependency.

Q109 How do you implement a reactive aggregator with a partial-failure fallback?

ANSWER

Fetch each source with its own timeout and error fallback so one failure yields a default rather than failing the whole aggregate, then combine. The response is always assembled, degraded where a source was unavailable.

```java
Mono<Inventory> inv = inventoryClient.get(id)
    .timeout(Duration.ofMillis(400))
    .onErrorReturn(Inventory.unknown());
Mono<Price> price = pricingClient.get(id)
    .timeout(Duration.ofMillis(400))
    .onErrorReturn(Price.unavailable());
return Mono.zip(inv, price).map(t -> new ProductView(t.getT1(), t.getT2()));
```

Q110 How do you consume a Kafka topic and expose it as a live SSE stream?

ANSWER

Bridge the reactive Kafka receiver into a multicast sink, then expose the sink's `Flux` as a server-sent-events endpoint. Clients receive topic events live, with the sink providing buffering and backpressure between the consumer and the HTTP subscribers.

```java
kafkaReceiver.receive()
    .doOnNext(r -> sink.tryEmitNext(r.value()))
    .subscribe();

@GetMapping(value="/stream", produces=MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<Event> stream() { return sink.asFlux(); }
```

Q111 How do you enforce a per-tenant rate limit reactively?

ANSWER

Key a reactive rate limiter (Resilience4j or a Redis-backed limiter at the gateway) by tenant id read from the request or reactive context, rejecting calls that exceed a tenant's quota with HTTP 429. This isolates tenants so one cannot exhaust shared capacity.

Q112 How do you guarantee an event is published only if the DB write commits?

ANSWER

Use the transactional outbox: write the domain change and an outbox row in the same reactive transaction, then a separate relay streams the outbox to the broker and marks rows sent. This avoids the dual-write problem — the event exists if and only if the data was committed.

Q113 How do you drain in-flight requests during a graceful shutdown?

ANSWER

On shutdown, stop accepting new connections, let the readiness probe fail so the load balancer removes the instance, and allow the reactive server a grace period to complete in-flight requests before terminating. This prevents dropped requests during rolling deployments and autoscaling.

© 2026 ByteMart Digital

Q114 How do you implement an idempotent reactive consumer?

ANSWER

Deduplicate on a stable message id before applying the effect — for example a conditional insert into a processed-ids table (or a unique constraint) inside the same transaction as the effect. If the id already exists, skip the effect and acknowledge, making reprocessing a no-op.

java

```
return processedRepo.existsById(msg.id())
    .flatMap(seen -> seen
        ? Mono.empty()                        // duplicate: skip
        : handle(msg).then(processedRepo.save(msg.id())))
    .then(ack(msg));
```

Q115 How do you chain reactive Saga steps with compensation cleanly?

ANSWER

Express each step as a `Mono` and attach its compensation via `onErrorResume` that runs the undo and re-raises, so a downstream failure unwinds the completed steps in reverse before propagating the error.

java

```
return reserveStock(order)
    .flatMap(r -> chargePayment(order)
        .onErrorResume(e -> releaseStock(order).then(Mono.error(e))))
    .flatMap(p -> ship(order)
        .onErrorResume(e -> refund(order)
            .then(releaseStock(order)).then(Mono.error(e))));
```

Q116 How do you apply a circuit breaker and retry together correctly?

ANSWER

Order matters: place the retry **inside** the circuit breaker so retries count toward the breaker's failure tally, and keep retries few with backoff so a struggling dependency is not hammered. When the breaker opens, calls fail fast to a fallback without retrying.

java

```
return call()
    .retryWhen(Retry.backoff(2, Duration.ofMillis(100)))
    .transformDeferred(CircuitBreakerOperator.of(cb))
    .onErrorResume(e -> Mono.just(fallback()));
```

© 2026 ByteMart Digital

© 2026 ByteMart Digital